Reinforcement Learning en Super Mario Bros

Urko Lopez Sacristan y Julen Larrañaga Karrera

Índice

1 Introducción y plantemiento del problema **1**
1.1 Espacio de estados 1
1.2 Espacio de acciones 1
1.3 Sistema de recompensas 2

2 Optimización **2**

3 Evaluación y comparación **3**

4 Discusión y conclusión **5**

1. Introducción y plantemiento del problema

El objetivo de este proyecto es entrenar un agente capaz de jugar de manera autónoma al primer nivel del juego Super Mario Bros utilizando técnicas de aprendizaje por refuerzo. Para ello, se utiliza el entorno proporcionado por la librería gym-super-mario-bros, que permite interactuar con el juego original emulado en NES-Py mediante la interfaz estándar de entornos de aprendizaje por refuerzo.

El agente interactúa con el entorno observando el estado de juego y seleccionando acciones discretas en cada paso temporal. El objetivo del agente es maximizar la recompensa acumulada durante cada episodio, aprendiendo una política que le permita avanzar por el nivel lo más rápido posible y sin tocar a enemigos.

1.1. Espacio de estados

El agente recibe como observación la imagen del estado acual del juego. Para reducir la complejidad del problema y facilitar el entrenamiento del modelo, las observaciones se preprocesan aplicando varias transformaciones. En primer lugar, las imágenes se convierten a escala de grises y se redimensionan a una resolución de 84x84 píxeles. Además, para capturar información temporal sobre el movimiento del personaje y los elementos del entorno, se utilizan los últimos cuatro frames consecutivos del juego, que se apilan formando un único estado de dimensión 4x84x84.

Este tipo de representación permite que la red neuronal del agente pueda inferir información sobre la velocidad y dirección del movimiento a partir de la secuencia de frames.

1.2. Espacio de acciones

El espacio de acciones del agente es discreto y se restringe utilizando el conjunto de acciones RIGHT_ONLY, que limita las posibles acciones a aquellas necesarias para progresar hacia la derecha en el nivel, descritas en la tabla 1. Esta reducción del espacio de acciones simplifica el problema de aprendizaje y facilita que el agente se concentre en estrategias relacionadas con el avance en el nivel.

Tabla 1: *Espacio de acciones RIGHT_ONLY de Super Mario Bros.*

Acción	0	1	2	3	4
Descripción	Nothing	Right	Right+jump	Right+run	Right+run+jump

1.3. Sistema de recompensas

El sistema de recompensas se basa principalmente en el progreso horizontal del agente. El agente recibe recompensas positivas proporcionales al desplazamiento, así como derrotar enemigos o recoger monedas, mientras que la pérdida de una vida termina el episodio y no otorga recompensa adicional. De este modo, el agente debe aprender una política que maximice la recompensa acumulada, equilibrando la exploración del entorno con la ejecución de acciones que le permiten avanzar de manera estable.

2. Optimización

El agente implementado sigue un enfoque Double DQN para aprender la política de control, utilizando una red neuronal convolucional que procesa los estados formados por frames apilados. La arquitectura está compuesta por tres capas convolucionales seguidas de dos capas fully connected que producen los valores Q para cada acción.

El entrenamiento utiliza experience replay mediante un replay buffer y una red target independiente que se sincroniza periodicamente con la red online. La política de exploración sigue un esquema ϵ-greedy con decaimiento.

La optimización de hiperparametros se realizó utilizando Optuna. Cada trial entrena un agente durante 500 episodios, utilizando la recompensa media de los episodios finales como función objetivo. Para mejorar la eficiencia objetivo se empleó TPESampler y MedianPruner, haciendo un total de 10 trials.

Los hiperparametros optimizados son los siguientes:

- **Learning rate**: optimización logarítmica uniforme en el rango [0.00001, 0.001]
- **Factor de descuento,** γ: optimización logarítmica uniforme en el rango [0.9, 0.999]
- **Epsilon decay**: optimizado en el rango [0.99999, 0.999999]
- **Batch size**: optimización catégorica entre [32, 64, 128, 256]
- **Target network sincronization rate**: optimización catégorica entre [5000, 10000, 20000]
- **Replay buffer capacity**: optimización catégorica entre [50000, 100000, 200000]

Los parámetros de exploración inicial y mínimo se mantuvieron constantes durante el proceso (eps = 0,9; eps_min = 0,1). Los parametros usados por defecto y los mejores se resumen en la tabla 2.

Tabla 2: *Valores de los hiperparámetros por defecto y óptimos usados.*

Parámetro	lr	γ	eps_dec	bat_size	sync_netw_rate	rep_buf_cap
Default	0,0001	0,99	0,9999975	256	10000	100000
Óptimos	5,6115e-05	0,9941	0,9999965	32	5000	100000

3. Evaluación y comparación

La evolución entre el agente base y el optimizado se realizó evaluando ambos modelos durante 30 episodios independientes. Durante la evaluación los modelos se ejecutaron sin entrenamiento adicional y con un valor de exploración bajo, eps = 0,05, para aplicar una política casi determinista.

Para cada episodio se registró la recompensa total acumulada, obteniendo una distribución de recompensas para cada agente. A partir de esos datos se calcularon estadísticas descriptivas para caracterizar el rendimiento de ambos modelos resumido en la tabla 3. También se han comparado ambas distribuciones mediante test Welch y Mann-Whitney U con nivel de significancia $\alpha = 0,05$. Estos resultados se reflejan en la tabla 4.

Tabla 3: *Resumen descriptivo del rendimiento por agente*

Agent	Mean	Std	Median	Min	Max
Default	2028.1	990.6	1915.5	233	3038
Óptimo	1942.6	690.5	2329.5	253	3010

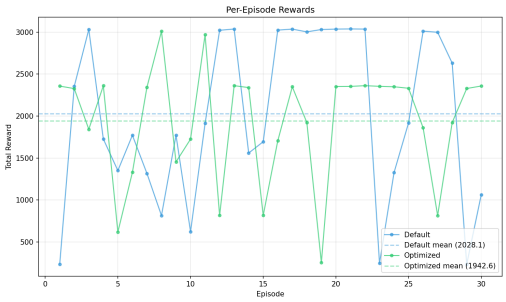
Tabla 4: *Pruebas estadísticas entre agente Default y Óptimo*

Test	Statistic	p-value
Welch t-test	0.388	0.6997
Mann–Whitney U	497	0.4917

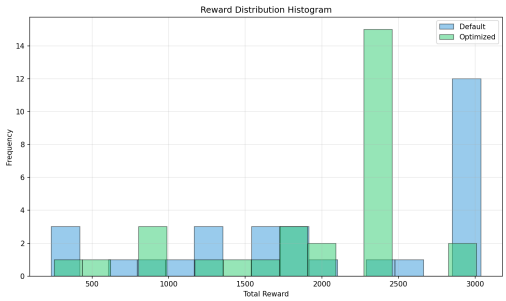
El agente base obtuvo una recompensa media ligeramente superior, aunque el agente optimizado mostró una variabilidad menor en sus resultados. Por otro lado, la mediana del agente optimizado es mayor, indicando que en la mayoria de sus episodios su rendimiento es más consistente.

En los test estadísticos los valores p son superiores al nivel de significancia establecido, por lo que no se observa una diferencia estadísticamente significativa entre ambos agentes. El tamaño calculado mediante Cohen's d = -0,10 confirma que la diferencia entre ambos modelos es muy pequeña.

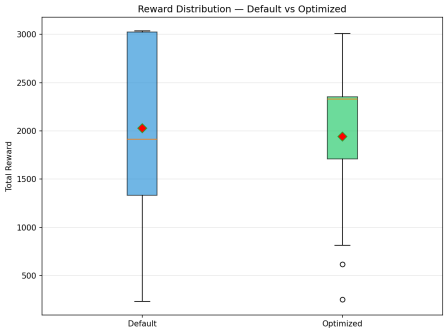
Para complementar el análisis se generaron visualizaciones de las distribuciones de recompensa, incluyendo boxplots, histogramas y recompensas por episodio reflejadas en la figura 1.



(a) *Episode rewards*



(b) *Histogram*



(c) *Boxplot*

Figura 1: *Comparación visual de métricas de rendimiento del agente.*

Los gráficos confirman visualmente los resultados estadísticos. En la figura 1c y la figura 1a se contempla que el agente base alcanza una media superior, mientras que el optimizado presenta menor variabilidad, reflejando mayor consistencia. En la figura 1b se aprecia que, aunque el agente base alcanza el final del nivel en más episodios, el optimizado consigue llegar más lejos en más episodios, mostrando mayor estabilidad. Ésto concuerda con la menor desviación estandar y la mediana más alta observada en la distribución de recompensas del agente óptimo.

4. Discusión y conclusión

Viendo los resultados obtenidos, concluimos que el entorno escogido es especialmente exigente para métodos de aprendizaje por refuerzo debido a la combinación de recompensas escasas, alta variabilidad en las transiciones y en la necesidad de coordinar acciones a largo plazo para avanzar niveles. Esto se refleja claramente en la optimización de hiperparámetros, en la que el número reducido de episodios de exploración limita la capacidad de aprendizaje, y sería necesario incrementar los episodios de entrenamiento para obtener mejoras significativas y sostenidas.

Por otro lado, no se han generado los gráficos de confianza ni de tolerancia durante el entrenamiento, ya que como se observa en la figura 2, el progreso a lo largo de los episodios es altamente irregular, debido a que el agente muere frecuentemente contra el primer enemigo, lo que impide representar de forma clara la clara evolución del rendimiento promedio.

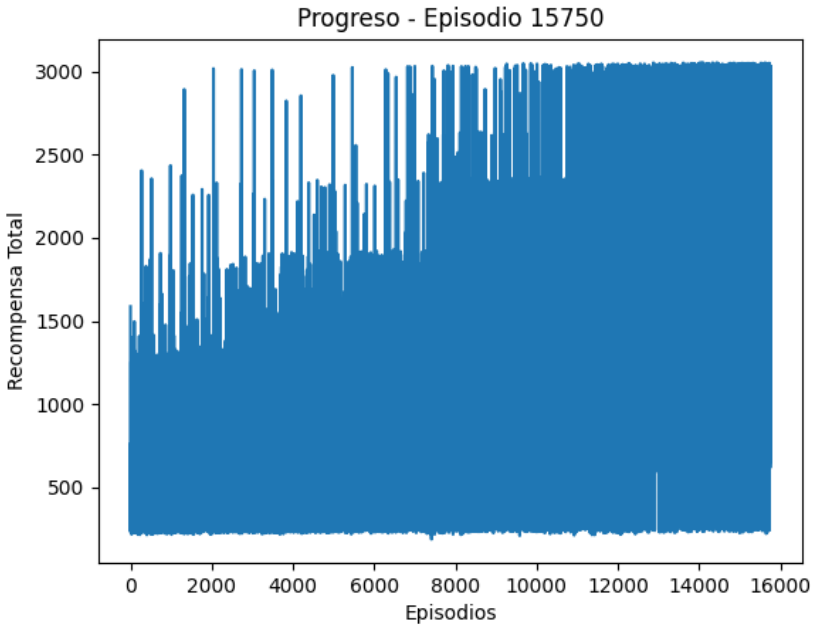


Figura 2: *Progreso de la recompensa por episodio durante el entrenamiento.*

Por último, después de obtener un agente con muy buen rendimiento, sería interesante ampliar el conjunto de acciones disponibles, teniendo en cuenta la posibilidad de desplazarse a la izquierda y agacharse, para así simular un comportamiento más humano. También se podría evaluar el desempeño del agente actual en niveles posteriores, midiendo así su capacidad de generalización (*Transfer Learning*).